



**Mondragon
Unibertsitatea**

**Escuela Politécnica
Superior**

Reinforcement Learning -RL-

Introduction

Aitor Duo Zubiaurre

aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)

T. +34 670 254 555

www.mondragon.edu

This presentation is based on:

Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction.
MIT Press, Cambridge, MA, 2018.

Benvenuto

Baie Welkom

Tervetuloa

Willkommen

Mwalandiridwa

ongi etorri

Bem-Vindos

Nazmukelekile

Welkam

Velkommen

Bienvenue

Bienvenido

Welcome

Welkom

Welcome

- Schedule**

	13 th july	14 th july	15 th july	16 th july	17 th july	
8:30-10:00	Lecture	Lecture	Lecture	Lecture		
10:00-10:15	<i>Break</i>	<i>Break</i>	<i>Break</i>	<i>Break</i>	10:00-13:00	Final presentations
10:15-13:00	Project work	Project work	Project work	Project work		
13:00-14:00	<i>Break</i>	<i>Break</i>	<i>Break</i>	<i>Break</i>		
14:00-16:30	Project work	Project work	Project work	Project work		

- Course material:**

https://github.com/aduo7/RL_EU4DUAL.git

Introduction

- **Group Project Guidelines**
- **Team Organization**
 - 3 people per group
 - **Work Division:** Strongly recommended to divide roles by algorithm types
 - (e.g., Student 1: Classic RL/Tile Coding, Student 2: Deep RL/DQN or PPO, Student 3: Hyperparameter Tuning & Statistical Significance).
 - **Project Scope Options**
 - **Option A:** Choose a Gymnasium Environment (Recommended)
 - » Focus on vector-based tracking environments (e.g., LunarLander-v3, Acrobot-v1).
 - » Fast training on standard laptop CPUs.
 - **Option B:** Develop Your Own Problem
 - » **Warning:** Keep action and state spaces discrete or low-dimensional. Do not get stuck debugging environment syntax!
 - **Deliverables**
 - **Clean, Documented Source Code:** Organized repository containing training scripts, Optuna studies, and evaluation files.
 - **Poster Presentation (20-25 min):** 15 min presentation + 5-10 min Q&A.
 - » Must include: Convergence plots, hyperparameter tuning results, and a statistical significance analysis (e.g., Welch's t-test) across multiple random seeds.

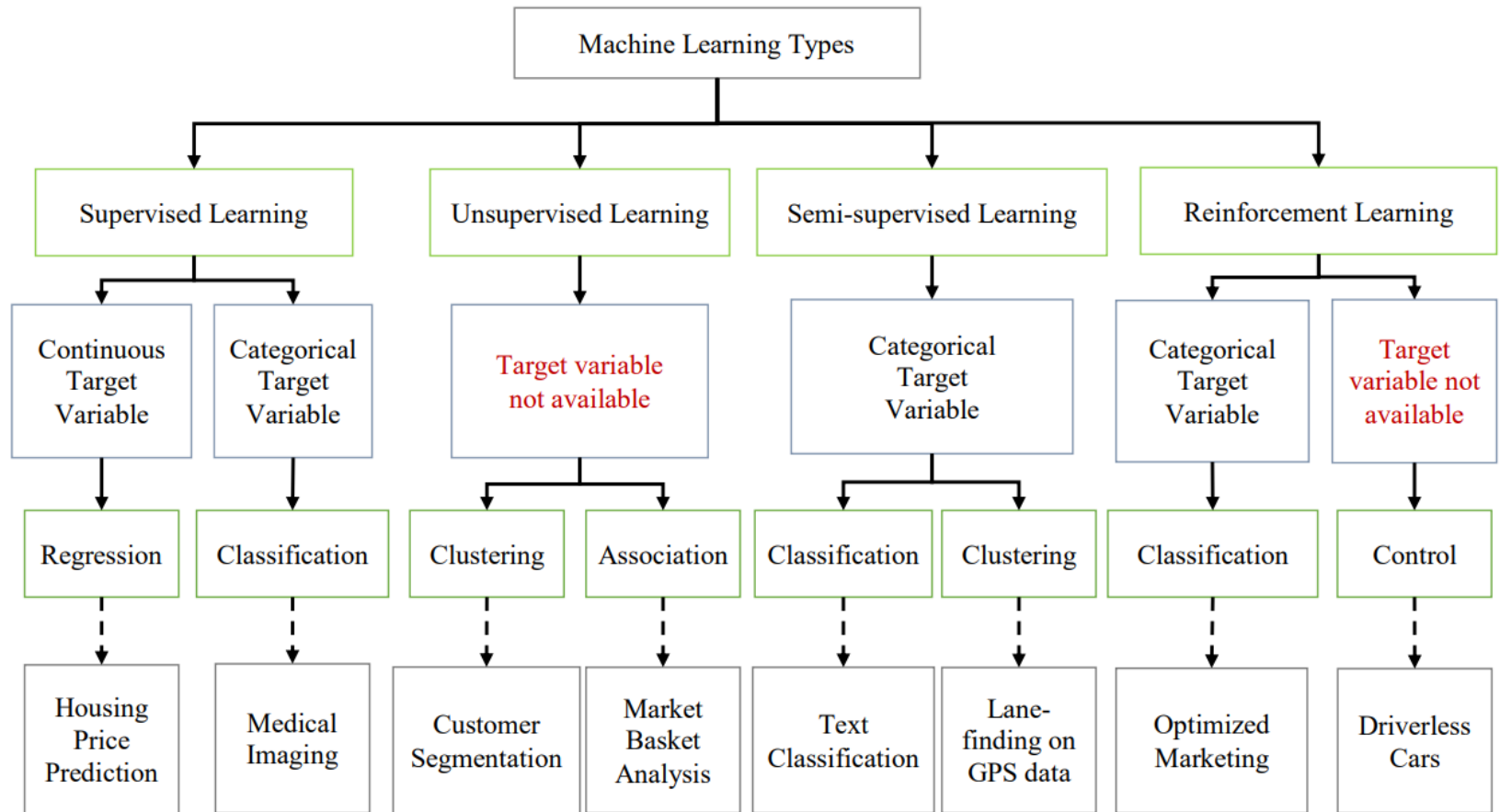
Introduction

- **Course Outline**

1. Introduction
 - General concepts
 - Gymnasium
2. RL Basics
 - Markov Decision Process
3. Classic RL
 - Temporal difference Learning
 - » Sarsa
 - » Q learning
4. Deep RL
 - DQN
 - PPO
5. Evaluation
 - Hyperparameter optimization
 - Evaluation metrics
 - Statistical significance tests

Introduction

- Machine Learning taxonomy**



Shyam, Radhey & Singh, Ria. (2021). A Taxonomy of Machine Learning Techniques. 8. 18-25.

Introduction

- Throughout our lives through interactions we gain knowledge about our environment ourselves.

Supervised Machine Learning

When you try to warn a kid to not touch fire because it will hurt!



Reinforcement Machine Learning

When the kid tries to touch the fire anyway and *learns* that it is not good to touch fire!



<https://thinkingneuron.com/data-science-interview-questions-reinforcement-ml/>

Introduction

Supervised Learning

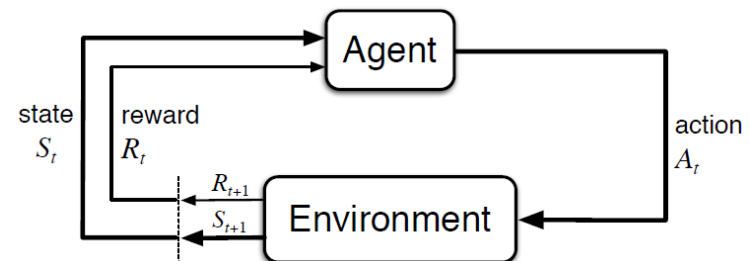
Data Distribution: i.i.d. Assumption Holds

- Data points (X_i, y_i) are Independent and Identically Distributed.
 - Sampling a row from a database does not alter the remaining rows.
- **Feedback Loop: Immediate & Dense**
 - Every prediction $\hat{y}_i$ receives an immediate evaluation via a loss function $\mathcal{L}(\hat{y}_i, y_i)$ against a fixed ground-truth label.

Reinforcement Learning

Data Distribution: i.i.d. is Violated (Sequential Dependence)

- Data is a trajectory
 $(\tau): s_0, a_0, r_1, s_1, a_1, r_2$
- Current data depends heavily on past data.
- The agent's action a_t directly forces a shift in the future data distribution:



Introduction

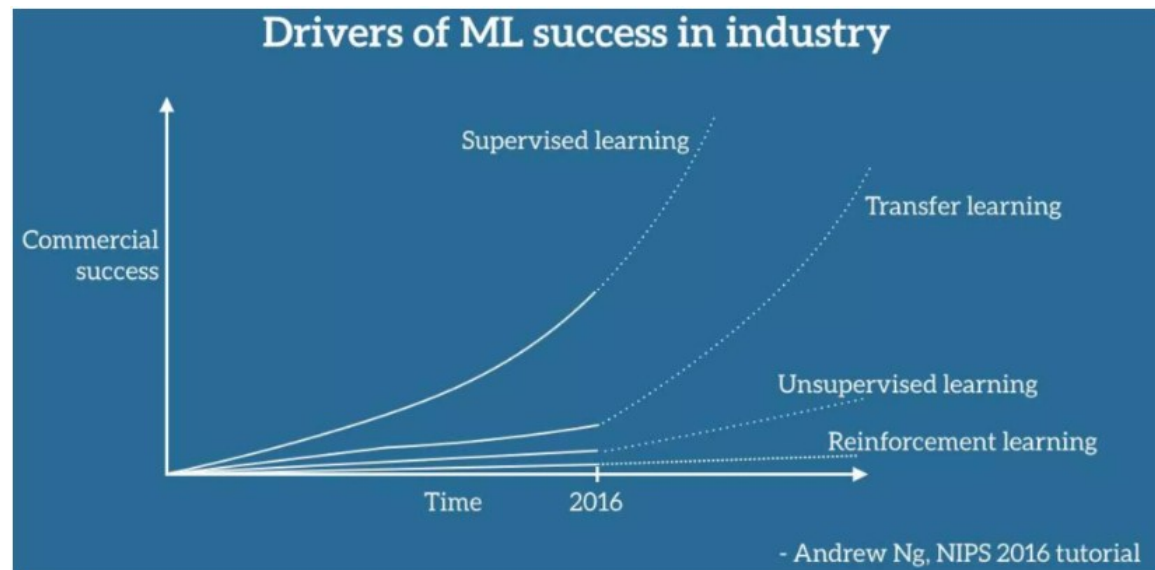
- **What distinguishes reinforcement learning from other machine learning methods?**
 - There is no supervision, only an indicator of reward, how good or bad the action taken has been.
 - Reward is delayed, not instantaneous.
 - The action taken at time t affects future rewards.

Introduction

- RL algorithms typically require millions of interactions to learn effective policies.
- While simulations can help with training, transferring policies from simulation to reality is challenging.
- Designing appropriate reward functions for industrial tasks is complex.
- Industries need consistent, predictable performance.
- Limited interpretability.

But be optimistic. We will see a gradual improvement next years.

- Applications where occasional failures are acceptable.
- Tasks where traditional approaches have failed.



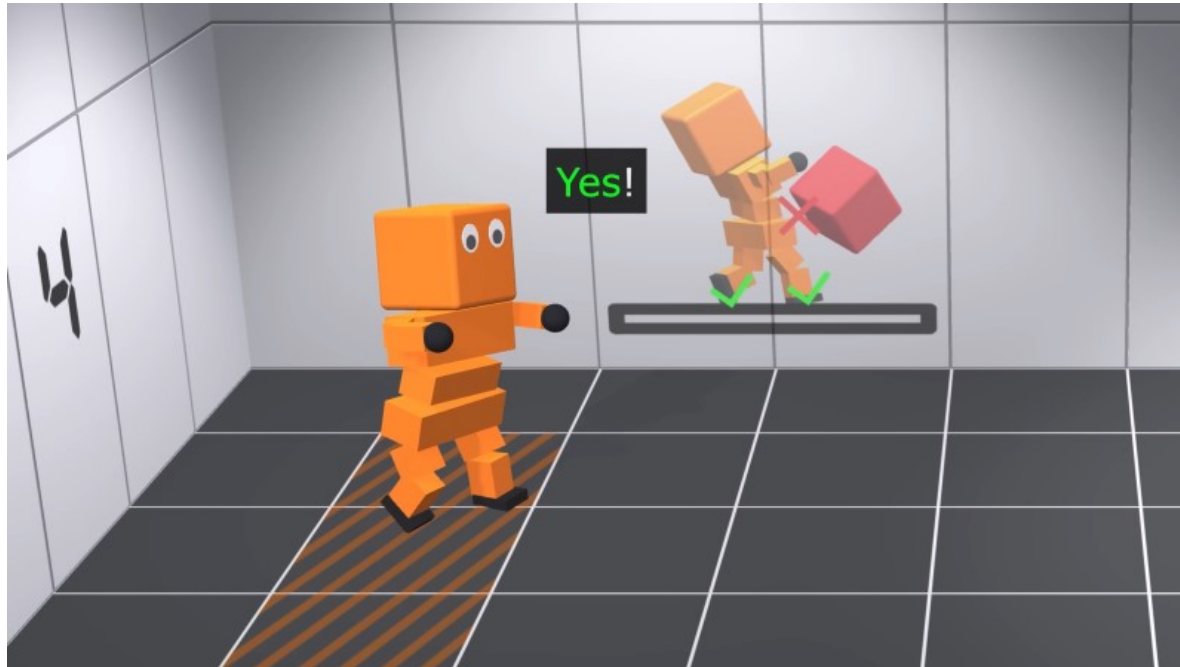
Introduction

- **Examples Reward or “RewArt”**
 - Autonomous driving:
 - +R reward for staying in the lane and maintaining a safe speed
 - -R reward for hitting obstacles or violating traffic rules
 - Robot arm in a factory setting
 - +R reward for successfully completing tasks (e.g., picking, placing objects)
 - -R reward for damaging objects or making errors in assembly
 - Customer service chatbot
 - +R reward for resolving customer inquiries efficiently and with high satisfaction
 - -R reward for misunderstandings or unsatisfactory responses

Introduction

- **Examples:**

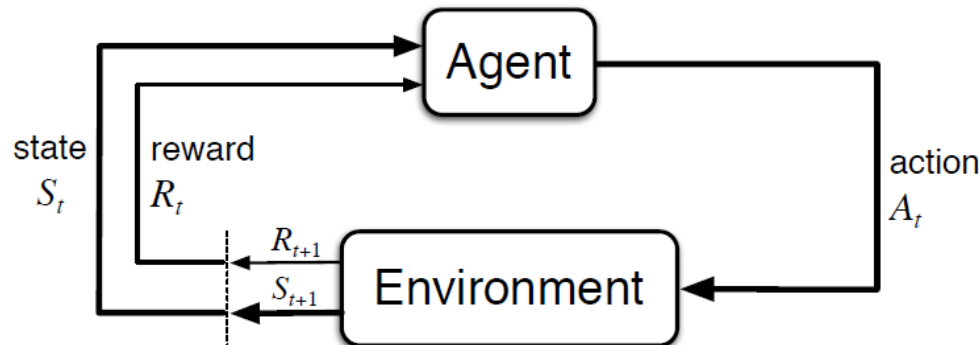
- https://www.youtube.com/watch?v=L_4BPjLBF4E



Introduction

- **The basics**

- **Agent:** interacts over time with its environment to achieve a goal.
- **Environment:** the dynamic landscape where an AI agent interacts and learns
- **Reward:** the agent is rewarded for taking controls that lead to successful states
- **State:** is a representation of the current environment that the agent is in
- **Action:** is a complete description of the state of the world



Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT press.

Introduction

- **The basics**
 - **Policy:** defines the learning agents way of behaving at a given time. Is a mapping from perceived states of the environment to actions to be taken when in those states.
 - Deterministic policy: $a = \pi(s)$; maps each state to a single action with certainty.
 - Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a \mid S_t = s]$; maps each state to a probability distribution over actions
 - **Value function:** Whereas the reward signal indicates what is good in an immediate sense, a value function specifies what is good in the long run. Roughly speaking, the value of a state is the total amount of reward an agent can expect to accumulate over the future.
 - Used to evaluate the goodness/badness of states, and therefore to select between actions

Introduction

- **The basics**

- **Model:** the model refers to a system or framework that simulates the environment's dynamics. It is used to predict the next state and the reward based on the current state and the chosen action.
 - **$\mathcal{P}$ Transition model / Dynamics model:** This predicts the next state s' given the current state s and action a . Describes the probability of transitioning to state s' when taking action a from state s .

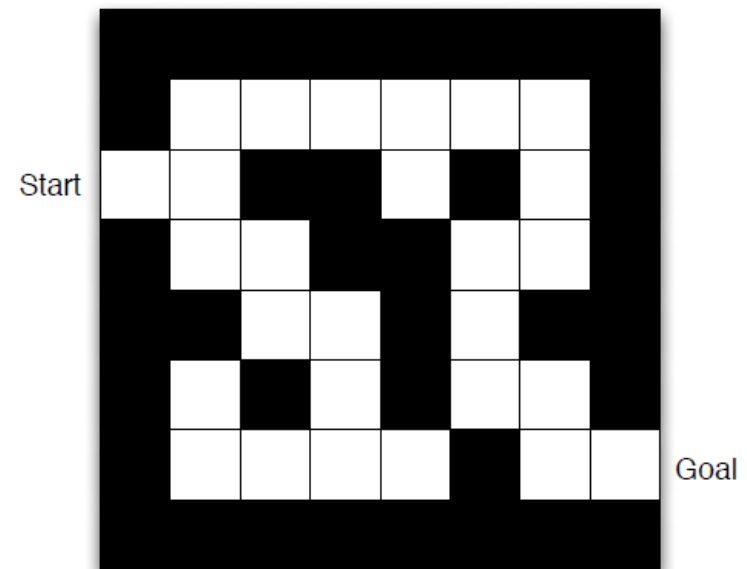
$$\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a]$$

- **$\mathcal{R}$ Reward model:** This predicts the reward r the agent will receive for taking action a in state s . Gives the expected immediate reward for taking action a in state s .

$$\mathcal{R}_s^a = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$$

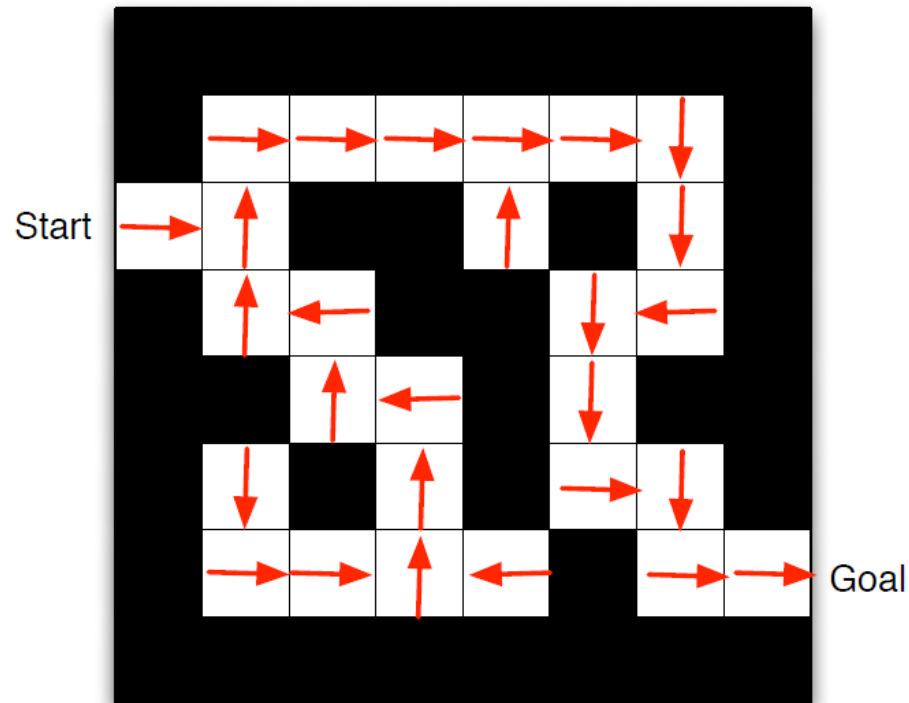
Introduction

- **Maze example**
 - Rewards: -1 per time step
 - Actions: N, E, S, W
 - States: agent location



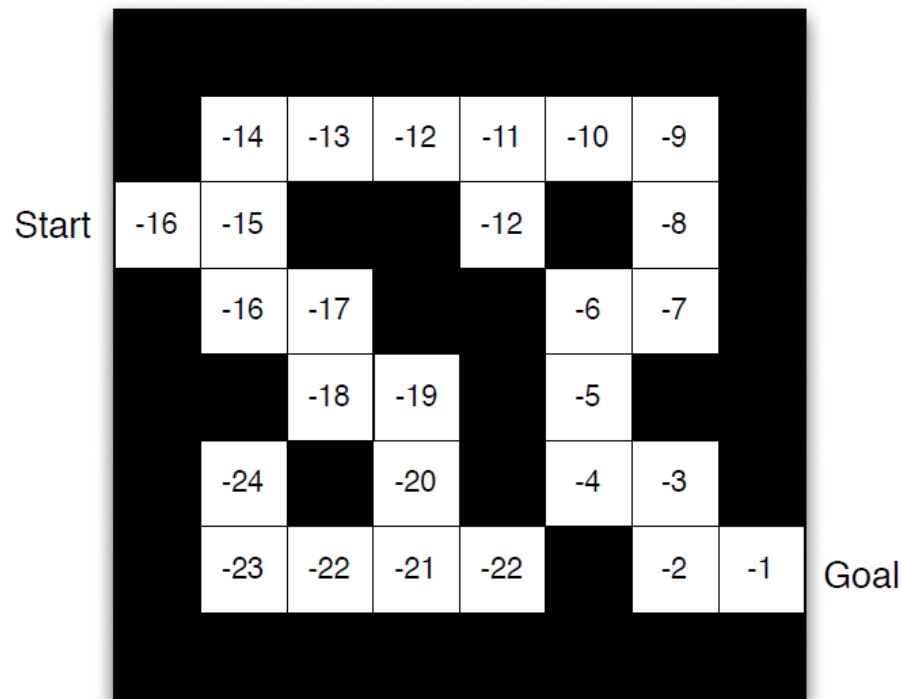
Introduction

- **Maze example: policy**
 - Arrows represent policy $\pi(s)$ for each state s



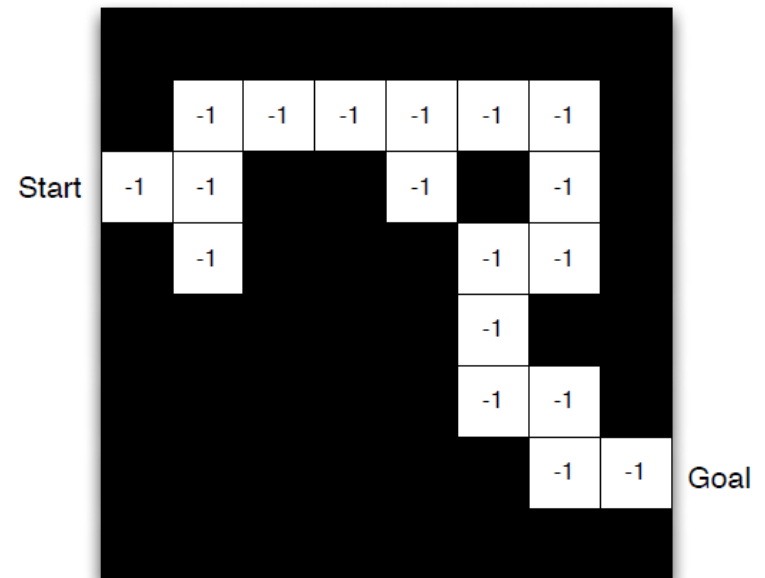
Introduction

- **Maze example: Value function**
 - Numbers represent value $v_{\pi}(s)$ of each state s



Introduction

- **Maze model**
 - Agent may have an internal model of the environment
 - Dynamics: how actions change the state
 - Rewards: how much reward from each state
 - The model may be imperfect
 - Grid layout represent transition model $\mathcal{P}_{ss'}^a$
 - Numbers represent immediate reward $\mathcal{R}_s^a$ from each state s (same for all a)



Introduction

- **Exploration vs exploitation**
 - Reinforcement learning is trial and error
 - The agent should discover a good policy from its experiences of the environment
 - **Exploration:** finds more information about the environment
 - **Exploitation:** exploits known information to maximize reward

It is important to explore as well as exploit

Introduction

- Examples
 - **Self-Driving Cars**
 - A self-driving car must decide on the best route to its destination.
 - **Exploration:** The car takes a new, less-used route to learn if it's faster or has less traffic.
 - **Exploitation:** The car consistently chooses the route it already knows to be the fastest based on historical data.

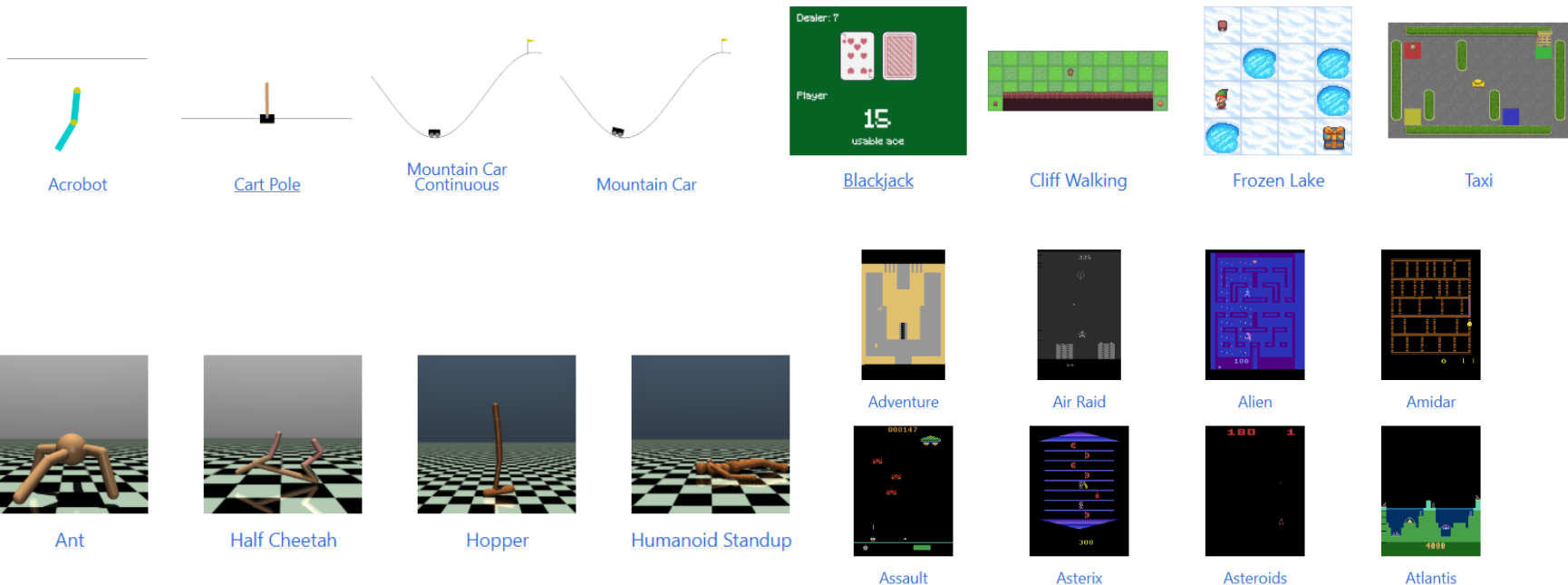
Introduction

- **Key aspects:**
 - The focus is on learning by interacting with an environment, like an opponent player.
 - The existence of a clearly defined objective is a prerequisite for the planning of correct behavior. Such planning must take into account the delayed effects of one's choices.
 - In contrast, tic-tac-toe has a relatively small, finite state set, whereas reinforcement learning can be used in scenarios where the state set is very large, or even infinite.

Introduction

• Gymnasium

- An API standard for reinforcement learning with a diverse collection of reference environments.
- `pip install gymnasium`
- <https://gymnasium.farama.org/>



Introduction

- **Frozen Lake**

- Frozen lake involves crossing a frozen lake from start to goal without falling into any holes by walking over the frozen lake.
- The game starts with the player at location [0,0]
- The player makes moves until they reach the goal or fall in a hole.

- **Action space:**

- 0: Move left
- 1: Move down
- 2: Move right
- 3: Move up

- **Rewards:**

- Reach goal: +1
- Reach hole: 0
- Reach frozen: 0

- **Episode End:**

- **Termination:**

- The player moves into a hole.
- The player reaches the goal at $\max(\text{nrow}) * \max(\text{ncol}) - 1$ (location $[\max(\text{nrow})-1, \max(\text{ncol})-1]$).

- **Truncation (when using the time_limit wrapper):**

- The length of the episode is 100 for 4x4 environment, 200 for FrozenLake8x8-v1 environment.



https://gymnasium.farama.org/environments/toy_text/frozen_lake/

Introduction

- **Exercise:**

- Look to gymnasium documentation and initialize the **FrozenLake-v1** environment and implement an agent that takes random actions until reaching a terminal state for 10 episodes/times.

```
import gymnasium as gym

# Initialise the environment
env = gym.make("FrozenLake-v1", render_mode="human")    render_mode = "rgb_array"

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)

for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

Lunar Lander simple example



Introduction

- General advice

Be patient and persistent

- Results may take **days to train**, and debugging RL models is harder than standard ML models.
- Focus on understanding **why** things work.

Once you see your agent learning and improving, its so satisfying!



**Mondragon
Unibertsitatea**

Escuela Politécnica
Superior

Eskerrik asko
Muchas gracias
Thank you

© 2026 Mondragon Unibertsitatea.

All rights reserved. This document and its contents are protected by intellectual property rights. Its total or partial reproduction, distribution, public communication, transformation, adaptation or creation of derivative works is expressly prohibited without the written consent of the owner.

Aitor Duo Zubiaurre

aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)

T. +34 670 254 555

www.mondragon.edu